

Instituto de
Computação
UNIVERSIDADE ESTADUAL DE CAMPINAS

PROCESSAMENTO DE LINGUAS NATURAIS - PROJETO 1

RECUPERAÇÃO / EXTRAÇÃO DE INFORMAÇÃO E GRAFO DE CONHECIMENTO

Daniel de Sousa Cipriano	ra 233228
Matheus Rufino da Silva	ra 221756
Gabriel Thomas Dias de Freitas	ra 235071
Diogo Barros de Paula	ra 242545
Eduardo dos Santos Neres	ra 188732

github.com/MatheuzRuf/projects-mc896.git

O SISTEMA SE DIVIDE EM SEIS ÁREAS PRINCIPAIS:

- Graph: implementação da estrutura do grafo de conhecimento.
- Preprocessing: pré-processamento do dataset.
- Boolean Retrieval: implementação de boolean retrieval e inverted index.
- Ranked Retrieval: implementação de ranked retrieval e TF-IDF.
- Vocabulary: definição de vocabulário.
- Information Extraction: extração de entidades e relações, para construção do grafo de conhecimento.

GRAPH - SCHEMA DO GRAFO DE CONHECIMENTO

- Seguimos o padrão de schema sugerido.
- Divisão entre nós de caso clínico e nós de atributos.
- Divisão entre relações de caso clínico e relações de atributos.

```
class NodeType(str, Enum):
    """Tipos de nós utilizados no Knowledge Graph."""

    PATIENT = "Patient"
    HISTORY = "History"
    SYMPTOM = "Symptom"
    FINDING = "Finding"
    EXAM = "Exam"
    EXAM_RESULT = "ExamResult"
    DIAGNOSIS = "Diagnosis"
    MEDICATION = "Medication"
    TREATMENT = "Treatment"
    OUTCOME = "Outcome"

    # Nós utilizados para decomposição de atributos
    MEASUREMENT = "Measurement"
    VALUE = "Value"
    UNIT = "Unit"
    REFERENCE_RANGE = "ReferenceRange"
    INTERPRETATION = "Interpretation"
    ANATOMICAL_SITE = "AnatomicalSite"
    STATUS = "Status"
    COURSE = "Course"

    # Ligação com vocabulários controlados
    VOCAB_CONCEPT = "VocabConcept"
```

```
class Relation(str, Enum):
    """Relações utilizadas no Knowledge Graph."""

    # Relações clínicas
    HAS_HISTORY = "HAS_HISTORY"
    PRESENTS_WITH = "PRESENTS_WITH"
    HAS_FINDING = "HAS_FINDING"
    UNDERWENT_EXAM = "UNDERWENT_EXAM"
    HAS_RESULT = "HAS_RESULT"
    REVEALS = "REVEALS"
    CONFIRMS = "CONFIRMS"
    EXCLUDES = "EXCLUDES"
    SUPPORTS = "SUPPORTS"
    PREDISPOSES_TO = "PREDISPOSES_TO"
    DIAGNOSED_WITH = "DIAGNOSED_WITH"
    TREATED_BY = "TREATED_BY"
    TARGETS = "TARGETS"
    HAS_OUTCOME = "HAS_OUTCOME"
    LEADS_TO = "LEADS_TO"

    # Relações de atributos
    HAS_DURATION = "HAS_DURATION"
    HAS_SIZE = "HAS_SIZE"
    LOCATED_IN = "LOCATED_IN"
    HAS_VALUE = "HAS_VALUE"
    HAS_UNIT = "HAS_UNIT"
    HAS_REFERENCE_RANGE = "HAS_REFERENCE_RANGE"
    HAS_LOW = "HAS_LOW"
    HAS_HIGH = "HAS_HIGH"
    HAS_INTERPRETATION = "HAS_INTERPRETATION"
    HAS_STATUS = "HAS_STATUS"
    HAS_COURSE = "HAS_COURSE"

    # Vocabulários controlados
    SAME_AS = "SAME_AS"
```

PREPROCESSING - TRATAMENTO INICIAL DO DATASET

- Geramos quatro saídas pós tratamento:
 - texto original do caso clínico.
 - texto normalizado (lowercase, unicode, etc), entrada para tokenização.
 - tokens gerados via regex focado na linguagem clínica.

```
# Ordem dos padrões é importante.
token_pattern = re.compile(
    r"""
    \d+(?:\.\d+)?%           # 92%, 12.5%
    |
    \d+(?:\.\d+)?(?:-\w+)?   # 5-day, 4, 850, 12.5
    |
    [a-zA-Z]+(?:/[a-zA-Z]+)+ # U/L, mg/L, kg/m2
    |
    [a-zA-Z]+(?:-[a-zA-Z]+)* # palavras e palavras-com-hífen
    |
    [<>]=?                  # <, >, <=, >=
    |
    [+-]\d+(?:\.\d+)?       # +5, -2.5
    """,
    re.VERBOSE,
)
```

BOOLEAN RETRIEVAL - INVERTED INDEX

- Utilizamos a lista de tokens de cada caso clínico, para implementar a inverted index utilizando lista ligada.
- Definimos um atributo de chave para cada nó da lista, construído desmembrando os "case_id", o que possibilitou controle de ordenação crescente.

DICIONÁRIO

```
+-----+  
| "fever" | --->  
+-----+
```

POSTINGS LIST (Lista Ligada Simples)

```
+-----+  
| InvertedIndexNode |  
+-----+  
| case_id: "PMC2817501_01" |  
| tf: 3 |  
+-----+  
| doc_key: (2817501, 1) |  
| next: [ponteiro] |  
+-----+
```

```
+-----+  
| InvertedIndexNode |  
+-----+  
| case_id: "PMC11722600_02" |  
| tf: 1 |  
+-----+  
| doc_key: (11722600, 2) |  
| next: [ponteiro] |  
+-----+
```

Ordem Crescente: (2817501, 1) < (11722600, 2)

BOOLEAN RETRIEVAL - STOPWORDS

- Utilizamos o cálculo do DF simples (document frequency), para filtrar stopwords dentro dos índices das listas.
- Índices considerados stopwords são mantidos na lista, mas são marcados com uma flag para identificação.

```
"""
Calcula IDF para cada termo e marca stopwords.

- `idf_threshold`: marca como stopwords termos com `idf < idf_threshold` (padrão = 0.1).
- `max_df_frac`: marca como stopwords termos com (`df / total_docs`) >= `max_df_frac` (padrão = 0.8).

Ao menos um dos thresholds deve ser fornecido; se ambos forem None,
apenas calcula e armazena os idf sem marcar stopwords.
"""
for term, index in inverted_index.items():
    index.compute_idf(total_docs)

if idf_threshold is None and max_df_frac is None:
    return

for term, index in inverted_index.items():
    df_frac = index.n / total_docs if total_docs > 0 else 0.0

    mark = False
    if idf_threshold is not None and index.idf < idf_threshold:
        mark = True
    if max_df_frac is not None and df_frac >= max_df_frac:
        mark = True

    index.mark_as_stopword(mark)

return inverted_index
```

BOOLEAN RETRIEVAL - ALGORITMOS AND, OR

- Implementamos os algoritmos de filtro booleano AND, OR e NOT (suportam operações lineares).
- O suporte a expressões booleanas arbitrárias com parênteses foi limitado, porque a avaliação de precedências exigiria a construção de um parser sintático.

```
answer = set()
current_a = index_a.postings_head if index_a else None
current_b = index_b.postings_head if index_b else None
while current_a is not None or current_b is not None:
    if current_a is not None and (current_b is None or current_a.doc_key < current_b.doc_key):
        answer.add(current_a.data[0]) # adiciona o case_id
        current_a = current_a.next
    elif current_b is not None and (current_a is None or current_b.doc_key < current_a.doc_key):
        answer.add(current_b.data[0]) # adiciona o case_id
        current_b = current_b.next
    else: # current_a.doc_key == current_b.doc_key
        answer.add(current_a.data[0]) # adiciona o case_id
        current_a = current_a.next
        current_b = current_b.next

return answer
```

```
answer = set()
current_a = index_a.postings_head if index_a else None
current_b = index_b.postings_head if index_b else None
while current_a is not None and current_b is not None:
    if current_a.doc_key == current_b.doc_key:
        answer.add(current_a.data[0]) # adiciona o case_id
        current_a = current_a.next
        current_b = current_b.next
    elif current_a.doc_key < current_b.doc_key:
        current_a = current_a.next
    else:
        current_b = current_b.next

return answer
```

BOOLEAN RETRIEVAL - ALGORITMO NOT

- O algoritmo do NOT realiza a comparação de chaves em relação a uma outra lista ligada que armazena todos os "case_id".
- O conjunto retornado é o complemento do termo filtrado, ou seja, todos os docs que não possuem o termo.

```
def NOT(inverted_index, term, cases_list) -> set[str]:  
    """  
    Operacao booleana NOT.  
    Retorna o complemento do conjunto de documentos contendo o termo"""  
    index = inverted_index.get(term)  
    current_term = index.postings_head if index else None  
    current_universe = cases_list.cases_head  
  
    answer = set()  
  
    while current_universe is not None:  
        if current_term is None or current_universe.doc_key < current_term.doc_key:  
            answer.add(current_universe.data)  
            current_universe = current_universe.next  
        elif current_universe.doc_key == current_term.doc_key:  
            current_universe = current_universe.next  
            current_term = current_term.next  
        else:  
            current_term = current_term.next  
  
    return answer
```

RANKED RETRIEVAL - TF - IDF

- Junto com os “case_id” de cada nó da inverted index, armazenamos também o número de ocorrências de cada termo por documento.
- Utilizamos essas métricas para calcular um score de TF-IDF para cada termo em cada documento.

```
def RankedSearch(vectorizedDocs):  
    ranking = []  
    query = np.array(vectorizedDocs["query"])  
    for nome, vetor in vectorizedDocs.items():  
        vetor = np.array(vetor)  
        norma = np.sqrt(np.sum(vetor ** 2))  
        vetor_normalizado = vetor/norma  
        if nome == "query":  
            continue  
        score = np.dot(query, vetor_normalizado)  
        ranking.append((score, nome))  
  
    ranking.sort(reverse=True)  
  
    return ranking
```

```
def TF_IDF(indexInvertido, totalDocuments, query):  
    documentNames = set()  
    for invertedIndex in indexInvertido.values():  
        documentNames.update(invertedIndex.get_postings())  
  
    result = {name: [] for name in documentNames}  
    result["query"] = []  
    if isinstance(query, str):  
        query = preprocess_text(query).retrieval_tokens  
  
    for palavra, invertedIndex in indexInvertido.items():  
        valores = invertedIndex.get_postings_with_frequencies()  
        frequencies = dict(valores)  
        numDoc = len(valores)  
  
        for nome_documento in documentNames:  
            frequencia = frequencies.get(nome_documento, 0)  
            result[nome_documento].append(TermFrequency(frequencia)*InverseDocumentFrequency(numDoc, totalDocuments))  
  
        result["query"].append(TermFrequency(query.count(palavra))*InverseDocumentFrequency(numDoc, totalDocuments))  
  
    return result  
  
def TermFrequency(termFrequency):  
    return math.log(1+termFrequency)  
  
def InverseDocumentFrequency(numberDocumentsHasTerm, totalDocuments):  
    return math.log(totalDocuments/numberDocumentsHasTerm)
```

VOCABULARY - VOCABULÁRIO BASE

- Para mapear as entidades do grafo de conhecimento, definimos planilhas de vocabulários.
- Cada entrada de vocabulário possui as colunas term, aliases, type, source e code.
- O vocabulário é compilado em um regex que casa texto com a representação no grafo.

- anatomical_sites.csv
- courses.csv
- diseases.csv
- exams.csv
- findings.csv
- interpretations.csv
- medications.csv
- statuses.csv
- symptoms.csv
- treatments.csv
- units.csv

```
@dataclass(frozen=True)
class VocabularyEntry:
    """
    Uma entrada de vocabulário usada pelo entity extraction.

    term:
        | Forma canônica do conceito.

    aliases:
        | Outras formas de escrita que devem ser reconhecidas.

    type:
        | Tipo do nó que será criado no knowledge graph.

    source:
        | Origem do conceito, por exemplo:
        | manual, MeSH, LOINC, etc.

    code:
        | Código externo, quando disponível.
    """

    term: str
    aliases: tuple[str, ...]
    type: str
    source: str = "manual"
    code: str = ""
```

INFORMATION EXTRACTION

- Relações clínicas são geradas automaticamente por regras tipadas e padrões de contexto.
- Negação, histórico e confirmação alteram a relação criada, evitando associações positivas indevidas.
- O pipeline integra entidades, medições e relações, exportando nodes.csv e edges.csv com integridade referencial.

```
NEGATION_PATTERN = re.compile(
    r"(?:\bno\b|\bwithout\b|\bdenied\b|\babsence of\b|\bnegative for\b)"
    r"(?:\s+\w+){0,5}\s*$",
    re.IGNORECASE,
)

HISTORY_PATTERNS = [
    "history of",
    "history includes",
    "past medical history of",
]

CONFIRM_PATTERNS = [
    "confirmed",
    "confirming",
]
```

```
def build_case_graph(
    case_id: str,
    case_text: str,
    vocabulary: list,
) -> tuple[list[Node], list]:
    patient = Entity(
        label="patient",
        type=NodeType.PATIENT.value,
        source="case",
    )
    extracted_entities = extract_entities(case_text, vocabulary)
    entities = [patient, *extracted_entities]
    measurements = extract_structured_measurements(
        case_text,
        extracted_entities,
    )

    node_map: dict[str, Node] = {}
    nodes: list[Node] = []

    for entity in entities:
        node = _entity_to_node(case_id, entity)
        if node.node_id not in node_map:
            node_map[node.node_id] = node
            nodes.append(node)

    edges = extract_case_relations(case_id, entities, case_text)

    measurement_nodes, measurement_edges = _build_measurement_graph(
        case_id,
        measurements,
    )
    for node in measurement_nodes:
        if node.node_id not in node_map:
            node_map[node.node_id] = node
            nodes.append(node)

    edges.extend(measurement_edges)
    edges = _dedupe_edges(edges)

    valid_edges = []
    for edge in edges:
        if edge.source_id in node_map and edge.target_id in node_map:
            valid_edges.append(edge)

    return nodes, valid_edges
```

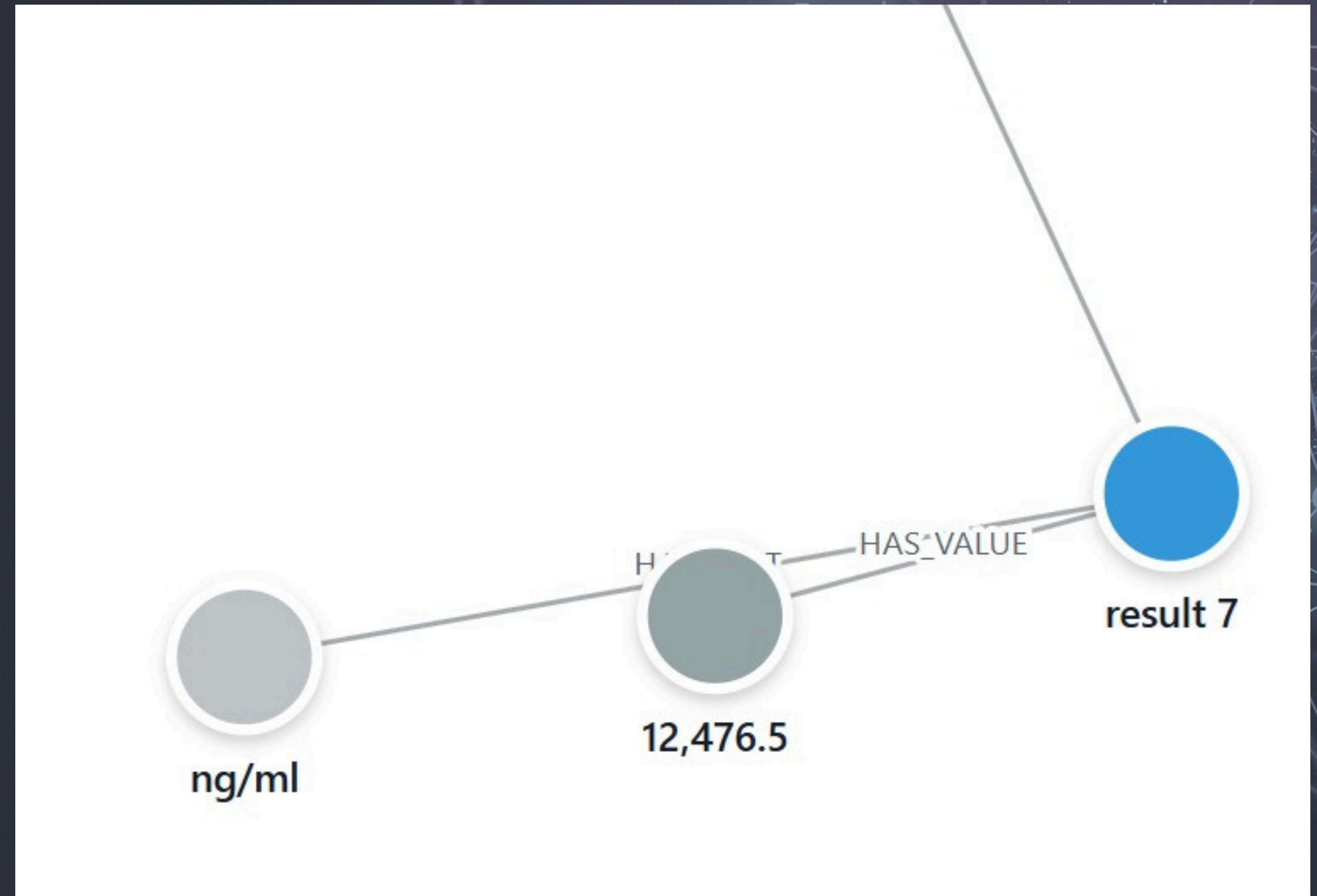
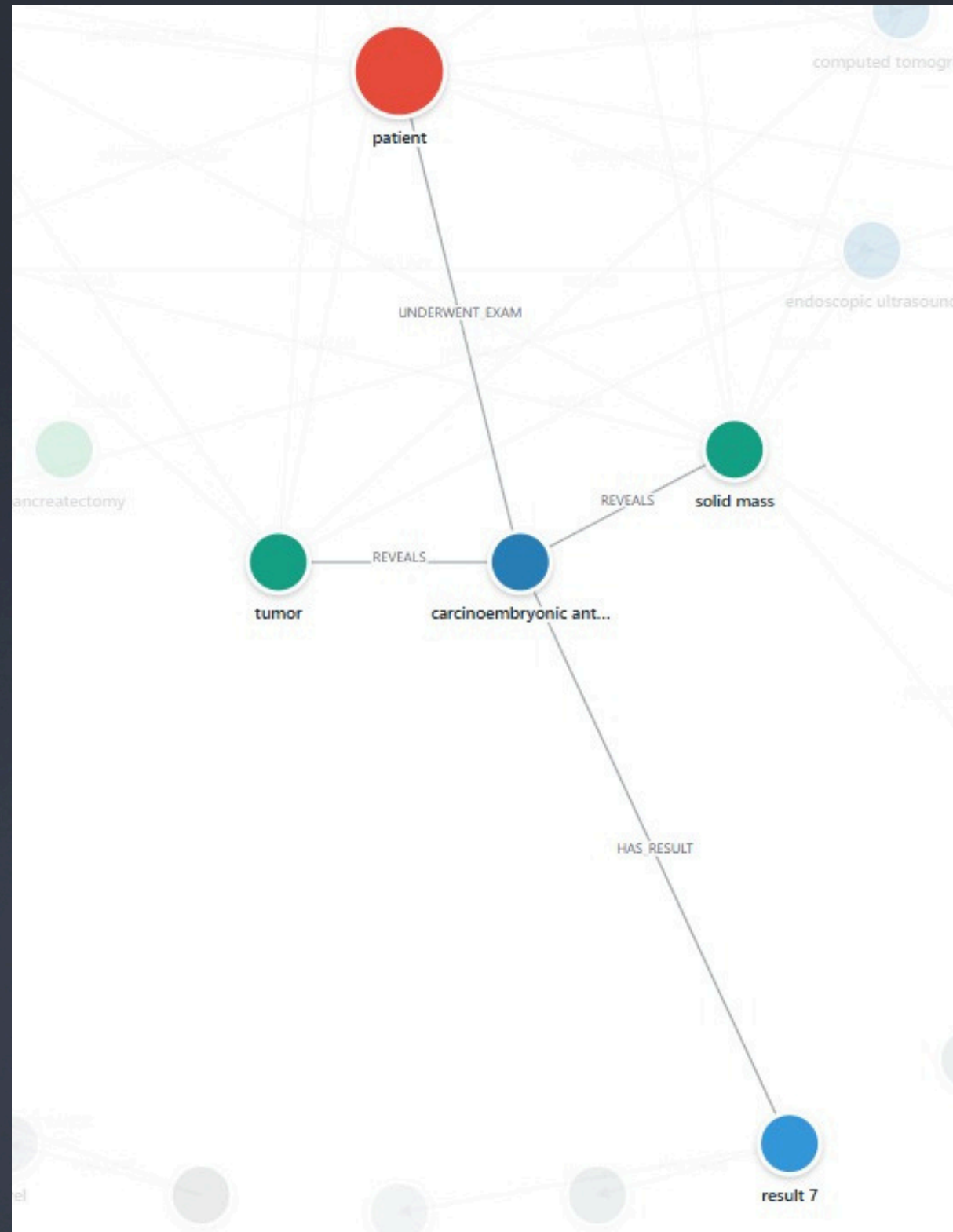
-O PARA



● AnatomicalSite ● Exam ● ExamResult ● Finding ● Interpretation ● Measurement ● Outcome ● Patient ● ReferenceRange ● Symptom ● Treatment ● Unit ● Value

● AnatomicalSite ● Exam ● ExamResult ● Finding ● Interpretation ● Measurement ● Outcome ● Patient ● ReferenceRange ● Symptom ● Treatment ● Unit ● Value

GRAFO DE CONHECIMENTO - EXEMPLO DE RELACIONAMENTO ENTRE ENTIDADES



INTERFACE GERAL - TEXTO, TOKENS, STOPWORDS

Inspecionar Caso

Texto original

A 35-year-old male with a 2-day history of chest tightness and fever was admitted to the emergency department on July 13, 2022. His medical history included lower extremity venous thrombosis, pulmonary nodules, cardiac surgery, and hypertension. Two days ago, he experienced significant dyspnea accompanied by intermittent fever and chills. His physical examination revealed

Texto normalizado

a 35-year-old male with a 2-day history of chest tightness and fever was admitted to the emergency department on july 13, 2022. his medical history included lower extremity venous thrombosis, pulmonary nodules, cardiac surgery, and hypertension. two days ago, he experienced significant dyspnea accompanied by intermittent fever and chills. his physical examination revealed

Tokens

the lower lobes of
both lungs figure 1
a and b the
patient was

Stopwords filtradas

with no the were
and in the and
in in a in the
and in the of 1

INTERFACE GERAL - MATRIZ TF-IDF, FILTROS

TF-IDF (termos × documentos)

	PMC10106591_01	PMC10405809_01	PM
35-year	0.000	0.000	0.0
male	0.000	0.000	0.9
admitted	0.961	0.000	0.0
intensive	0.000	0.000	0.0
care	1.128	0.000	0.0
unit	0.961	0.000	0.0
through	0.000	0.000	0.0
accident/ emergency	0.000	0.000	0.0
after	0.370	0.233	0.0

Consulta

ex: fever AND cough or 'fever'

Ranked (tf-idf)



Buscar

Consulta

ex: fever AND cough or 'fever'

Boolean (AND/OR/NOT)



Buscar